

Fetch API

The **Fetch API** is the modern replacement for XMLHttpRequest.

It allows your frontend (React / Vanilla JS / Next.js etc.) to communicate with servers (Node / Express / APIs).

Think of **fetch()** = **Promise-based HTTP client built into the browser.**

Basic Syntax

```
fetch(url, options);
```

Parameter	Meaning
url	API endpoint you want to call
options	Method, headers, body, etc. (optional)

Fetch **always** returns a **Promise**.

Simple GET Request

```
fetch("https://jsonplaceholder.typicode.com/users")  
  .then((response) => response.json())  
  .then((data) => console.log(data))  
  .catch((error) => console.error(error));
```

Request → Response Object → Convert to JSON → Use Data

Fetch does **NOT** directly give data.

It gives a **Response object**.

```
fetch(url).then(res => {  
  console.log(res);  
});
```

Property	Meaning
res.ok	true if status is 200–299
res.status	HTTP status code
res.headers	Response headers
res.json()	Convert body → JS object
res.text()	Convert body → text

`res.json()` is **also async** → returns Promise.

```
const data = await res.json();
```

Using Async/Await (Professional Way)

```
async function getUsers() {  
  try {  
    const response = await fetch("https://jsonplaceholder.typicode.com/users");  
  
    if (!response.ok) {  
      throw new Error("API Failed");  
    }  
  
    const data = await response.json();  
    console.log(data);  
  }  
}
```

```
} catch (error) {  
  console.error("Error:", error);  
}  
}
```

```
getUsers();
```

POST Request (Sending Data to Server)

```
async function addUser() {  
  const user = {  
    name: "Prajwal",  
    role: "Developer"  
  };  
  
  const response = await fetch("https://api.example.com/users", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify(user)  
  });  
  
  const data = await response.json();  
  console.log(data);  
}
```

Fetch Options Object (Real Power)

```
fetch(url, {  
  method: "GET | POST | PUT | DELETE",  
  headers: {},  
  body: JSON.stringify(data),  
  credentials: "include",  
  mode: "cors"  
});
```

Option	Use
method	HTTP method
headers	Send metadata
body	Data you send
credentials	Send cookies
mode	CORS handling

PUT / PATCH Request (Update Data)

```
await fetch("/users/1", {  
  method: "PUT",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({ name: "Updated Name" })  
});
```

DELETE Request

```
await fetch("/users/1", {  
  method: "DELETE"  
});
```

Fetch **ONLY** rejects on network failure NOT on 404 or 500.

```
try {  
  await fetch("/wrong-url");  
} catch {} Wrong way
```

✅ Correct Way:

```
const res = await fetch("/wrong-url");
```

```
if (!res.ok) {  
  throw new Error(`HTTP Error: ${res.status}`);  
}
```

Sending Query Parameters

```
fetch(`/products?category=food&page=2`);
```

Dynamic way:

```
const params = new URLSearchParams({  
  category: "food",  
  page: 2  
});
```

```
fetch(`/products?${params}`);
```

Fetch With Authorization (JWT — VERY COMMON IN MERN)

```
fetch("/profile", {  
  headers: {  
    "Authorization": `Bearer ${token}`  
  }  
});
```

Parallel API Calls (Performance Trick)

```
const [users, posts] = await Promise.all([  
  fetch("/users").then(res => res.json()),  
  fetch("/posts").then(res => res.json())  
]);
```

Abort / Cancel Request (Advanced)

```
const controller = new AbortController();

fetch("/data", { signal: controller.signal });

controller.abort(); // cancel request
```

1 What is Fetch API?

Answer:

Fetch API is a modern, promise-based JavaScript interface used to make HTTP requests from the browser to a server.

It replaces XMLHttpRequest and provides a cleaner, more flexible way to handle async network calls.

2 What does fetch() return?

Answer:

fetch() returns a Promise that resolves to a Response object, not the actual data.

3 Why do we need res.json()?

Answer:

Because Fetch does not automatically parse the response body.

4 Does Fetch reject on HTTP errors (404, 500)?

Answer: ✗ No

Fetch only rejects on network failure, not HTTP errors.

5 Why is res.json() asynchronous?

Answer:

Because the response body is a Readable Stream, and parsing happens asynchronously.

That's why this is required:

1 5 What problems can Fetch cause if misused?

- **Multiple unnecessary re-renders (React)**
- **No request cancellation → memory leaks**
- **No timeout support by default**
- **Manual error handling required**
- **Can cause waterfall requests if not parallelized**

Why do we need `JSON.stringify()` before sending body?

👉 Short Answer:

Because HTTP can only send text (or binary) — it cannot send JavaScript objects directly.

`fetch()` does NOT know how to convert your JS object into a transferable format.